

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: Mohammed Almas

Roll No.: A24126552098

Date: 25-08-2026

1. TITLE

Development of a Dynamic To-Do List Using JavaScript DOM Manipulation and Event Listeners

2. AIM

To develop a dynamic To-Do List application using HTML, CSS and JavaScript, where users can add tasks, mark tasks as completed, and delete tasks using DOM manipulation and event listeners.

3. OBJECTIVE

The objectives of this experiment are:

- To create a To-Do List webpage using HTML.
 - To apply CSS styling to the webpage.
 - To accept tasks using a text input field.
 - To dynamically create task elements using JavaScript.
 - To use `addEventListener()` for handling button clicks.
 - To implement a **Complete** functionality.
 - To implement a **Delete** functionality.
 - To use CSS classes dynamically.
 - To display a message when the task list is empty.
 - To understand JavaScript DOM manipulation.
-

4. THEORY

A. DOM Manipulation

DOM stands for Document Object Model. It represents the HTML document as objects that JavaScript can access and modify.

In this program, JavaScript dynamically creates:

- `<div>` for each task
 - `<span>` for task text
 - `<button>` for Complete
 - `<button>` for Delete
-

B. Event Listener

An event listener allows JavaScript to perform an action when a particular event occurs.

The following statement is used:

```
addButton.addEventListener("click", function() {
```

When the Add Task button is clicked, the function is executed.

Separate event listeners are also added to the Complete and Delete buttons.

C. `createElement()`

The `createElement()` method is used to dynamically create HTML elements.

Example:

```
let task = document.createElement("div");
```

This creates a new `<div>` element.

D. `classList.toggle()`

The `classList.toggle()` method adds or removes a CSS class.

In this program:

```
text.classList.toggle("completed");
```

adds the completed class when the task is incomplete and removes it when the task is already completed.

The .completed CSS class applies:

```
text-decoration: line-through;
```

```
color: gray;
```

Therefore, a completed task appears crossed out and gray.

E. remove()

The remove() method removes an element from the webpage.

```
task.remove();
```

This is used when the Delete button is clicked.

5. ALGORITHM / PROCEDURE

1. Create an HTML document.
2. Add a heading named **My To-Do List**.
3. Create a text input field for entering tasks.
4. Create an **Add Task** button.
5. Create a container to display tasks.
6. Display **No tasks available** initially.
7. Access the required HTML elements using getElementById().
8. Add a click event listener to the Add Task button.
9. Read the task entered by the user.
10. Check whether the input is empty.
11. Hide the empty message when a task is added.
12. Dynamically create a task container.
13. Create a span to store the task text.
14. Create a **Complete** button.
15. Add an event listener to the Complete button.
16. Apply or remove the completed CSS class.
17. Create a **Delete** button.
18. Add an event listener to the Delete button.
19. Remove the selected task when Delete is clicked.

20. Check whether any tasks remain.
 21. Display **No tasks available** if the list becomes empty.
 22. Clear the input field after adding a task.
 23. Verify the output in the browser.
-

6. PROGRAM

Todo_list.html

```
<!DOCTYPE html>
<html>

<head>

  <title>My To-Do List</title>

  <style>

    body {
      font-family: Arial;
      background-color: #f2f2f2;
      text-align: center;
      padding: 30px;
    }

    input {
      padding: 10px;
      width: 250px;
    }

    button {
      padding: 10px 15px;
      margin: 5px;
      cursor: pointer;
    }

    #taskList {
      width: 400px;
```

```
        margin: 20px auto;
    }

    .task {
        background-color: white;
        padding: 10px;
        margin: 10px;
        border-radius: 5px;
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .completed {
        text-decoration: line-through;
        color: gray;
    }

    #emptyMessage {
        color: gray;
    }

</style>

</head>

<body>

    <h1>My To-Do List</h1>

    <input
        type="text"
        id="taskInput"
        placeholder="Enter a task..."
    >

    <button id="addButton">
        Add Task
    </button>
```

```
<div id="taskList">
```

```
  <p id="emptyMessage">
```

```
    No tasks available.
```

```
  </p>
```

```
</div>
```

```
<script>
```

```
  let taskInput =
```

```
    document.getElementById("taskInput");
```

```
  let addButton =
```

```
    document.getElementById("addButton");
```

```
  let taskList =
```

```
    document.getElementById("taskList");
```

```
  let emptyMessage =
```

```
    document.getElementById("emptyMessage");
```

```
  addButton.addEventListener("click", function() {
```

```
    let taskText = taskInput.value;
```

```
    if (taskText.trim() === "") {
```

```
      return;
```

```
    }
```

```
    // Remove empty message
```

```
    emptyMessage.style.display = "none";
```

```
// Create task container

let task = document.createElement("div");

task.className = "task";


// Create task text

let text = document.createElement("span");

text.textContent = taskText;


// Create Complete button

let completeButton =
  document.createElement("button");

completeButton.textContent = "Complete";


// Complete task

completeButton.addEventListener(
  "click",
  function() {

    text.classList.toggle("completed");

  }
);


// Create Delete button

let deleteButton =
  document.createElement("button");
```

```
deleteButton.textContent = "Delete";

// Delete task

deleteButton.addEventListener(
  "click",
  function() {

    task.remove();

    // Show message if no tasks remain

    if (
      taskList.querySelectorAll(".task").length === 0
    ) {

      emptyMessage.style.display = "block";

    }

  }
);

// Add elements to task

task.appendChild(text);

task.appendChild(completeButton);

task.appendChild(deleteButton);

// Add task to list

taskList.appendChild(task);
```



```

        // Clear input

        taskInput.value = "";

    });

</script>

</body>

</html>

```

7. CODE EXPLANATION

Code	Purpose
<input>	Accepts the task from the user.
id="taskInput"	Identifies the task input field.
Add Task button	Adds a new task.
getElementById()	Selects HTML elements.
.value	Gets the entered task.
trim()	Removes extra spaces while checking empty input.
addEventListener()	Handles user actions.
createElement()	Creates HTML elements dynamically.
className	Assigns a CSS class to the task.
textContent	Sets the task text.
appendChild()	Adds elements to the webpage.
classList.toggle()	Adds/removes the completed class.
task.remove()	Deletes a task.
querySelectorAll()	Finds all task elements.
style.display	Shows or hides the empty message.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open webpage	To-Do List should appear	Appeared	Pass
TC-02	Click Add Task without entering text	Nothing should be added	No task added	Pass
TC-03	Enter "Complete Assignment"	Task should be accepted	Accepted	Pass
TC-04	Click Add Task	New task should appear	Appeared	Pass
TC-05	Click Complete	Task should be crossed out	Crossed out	Pass
TC-06	Click Complete again	Completed style should be removed	Removed	Pass
TC-07	Click Delete	Selected task should disappear	Deleted	Pass
TC-08	Delete all tasks	Empty message should appear	Appeared	Pass

9. OUTPUT

Output :

10. RESULT

Thus, a **dynamic To-Do List application** was successfully developed using **HTML, CSS and JavaScript**.

The application successfully allows the user to **add tasks, mark tasks as completed, delete tasks**, and display an **empty-list message** when there are no tasks. DOM manipulation and event listeners were successfully implemented.
